MIniproject

October 21, 2026

[2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

plt.rcParams.update({"figure.figsize": (7, 4), "axes.grid": True, "grid.alpha":␣
 ↪0.3})

df = pd.read_csv("smart_manufacturing_data.csv", parse_dates=["timestamp"])
print("Shape:", df.shape)
print("Missing:", df.isna().sum().sum(), "| Duplicates:", df.duplicated().sum())
df[["temperature", "vibration", "humidity", "pressure", "energy_consumption"]].
 ↪describe().round(2)
```

```
Shape: (100000, 13)
Missing: 0 | Duplicates: 0
```

[2]:

	temperature	vibration	humidity	pressure	energy_consumption
count	100000.00	100000.00	100000.00	100000.00	100000.00
mean	75.02	50.01	55.00	3.00	2.75
std	10.03	14.99	14.44	1.15	1.30
min	35.55	-17.09	30.00	1.00	0.50
25%	68.27	39.97	42.52	2.00	1.63
50%	75.06	49.96	54.98	3.01	2.74
75%	81.75	60.10	67.50	4.00	3.87
max	121.94	113.80	80.00	5.00	5.00

Fundamentals of Probability

A. a) Random Experiment, Sample Events & Sample space

[3]:
```
N = len(df)
df["A"] = df["anomaly_flag"] == 1             # anomaly
df["H"] = df["temperature"] > 85              # high temperature
df["M"] = df["maintenance_required"] == 1     # maintenance required

print("A and A' mutually exclusive :", (df.A & ~df.A).sum() == 0)
```

```
print("A and H mutually exclusive  :", (df.A & df.H).sum() == 0, f"({(df.A & df.
↪H).sum()} records in both)")
print("A is a subset of M          :", (df.A & ~df.M).sum() == 0)
```

```
A and A' mutually exclusive : True
A and H mutually exclusive  : False (6973 records in both)
A is a subset of M          : True
```

A .b) BAsic Probability rules

[6]:
```
# relative frequency
P = lambda e: e.mean()
pA, pH, pM, pAH = P(df.A), P(df.H), P(df.M), P(df.A & df.H)
print(f"P(A) = {pA:.4f}  P(H) = {pH:.4f}  P(M) = {pM:.4f}  P(A H) = {pAH:.4f}")

# complement rule
print(f"1 - P(A)     = {1 - pA:.4f}")
print(f"Direct P(A') = {P(~df.A):.4f}")

# adition theorem
print(f"P(A) + P(H) - P(A H) = {pA + pH - pAH:.4f}")
print(f"Direct P(A H)        = {P(df.A | df.H):.4f}")

# Multiplication Theorem
pH_given_A = P(df.H[df.A])
print(f"P(A)·P(H|A) = {pA * pH_given_A:.4f} | Direct P(A H) = {pAH:.4f}")
print(f"Independence check: P(A)·P(H) = {pA * pH:.4f} vs P(A H) = {pAH:.4f}")
```

```
P(A) = 0.0892  P(H) = 0.1596  P(M) = 0.1970  P(A H) = 0.0697
1 - P(A)     = 0.9108
Direct P(A') = 0.9108
P(A) + P(H) - P(A H) = 0.1791
Direct P(A H)        = 0.1791
P(A)·P(H|A) = 0.0697 | Direct P(A H) = 0.0697
Independence check: P(A)·P(H) = 0.0142 vs P(A H) = 0.0697
```

A .c) Conditional Probability & Baye's Theorem

[7]:
```
# Conditional Probability
pA_given_H = P(df.A[df.H])
pM_given_A = P(df.M[df.A])
print(f"P(Anomaly | High temp)   = {pA_given_H:.4f}  (P(A) = {pA:.4f})")
print(f"P(Maintenance | Anomaly) = {pM_given_A:.4f}")

#Bayes Theorem
likelihood = pH_given_A                                    # P(H|A)
evidence = likelihood * pA + P(df.H[~df.A]) * (1 - pA)     # P(H), total␣
↪probability
```

```
posterior = likelihood * pA / evidence
print(f"Prior = {pA:.4f} | Likelihood = {likelihood:.4f} | Evidence = {evidence:
  ↪.4f}")
print(f"Posterior P(A|H) = {posterior:.4f}  (direct = {pA_given_H:.4f})")
```

```
P(Anomaly | High temp)   = 0.4368  (P(A) = 0.0892)
P(Maintenance | Anomaly) = 1.0000
Prior = 0.0892 | Likelihood = 0.7821 | Evidence = 0.1596
Posterior P(A|H) = 0.4368  (direct = 0.4368)
```

PART B:- Discrete Random Variables & Functions

a) Bernoulli Distribution

[11]:
```
# Binary Variable & Probability of success
x = df["anomaly_flag"]
p = x.mean(); q = 1 - p
print(f"p = {p:.4f} | 1 - p = {q:.4f}")


# Mean & Variance
print(f"Mean = p = {p:.4f}       (sample = {x.mean():.4f})")
print(f"Var = p(1-p) = {p*q:.4f}   (sample = {x.var(ddof=0):.4f})")
```

```
p = 0.0892 | 1 - p = 0.9108
Mean = p = 0.0892       (sample = 0.0892)
Var = p(1-p) = 0.0812   (sample = 0.0812)
```

b) Binomial Distribution

[10]:
```
# PMF & CDF probabilities
n, k = 20, 2
X = stats.binom(n, p)
print(f"P(X = {k}) = {X.pmf(k):.4f}")
print(f"P(X <= 1) = {X.cdf(1):.4f}")
print(f"P(X >= 4) = {1 - X.cdf(3):.4f}")

# Mean, variance & empirical check
print(f"Mean np = {n*p:.3f} | Var np(1-p) = {n*p*q:.3f}")
counts = df["anomaly_flag"].groupby(np.arange(N) // n).sum()
print(f"Empirical: mean = {counts.mean():.3f}, var = {counts.var(ddof=0):.3f}")
```

```
P(X = 2) = 0.2812
P(X <= 1) = 0.4569
P(X >= 4) = 0.0967
Mean np = 1.783 | Var np(1-p) = 1.624
Empirical: mean = 1.783, var = 1.597
```

c) Poisson Distribution

[13]:
```
# event count variable and rate lambda
hourly = df.set_index("timestamp")["anomaly_flag"].resample("h").agg(["sum","count"])
hourly = hourly.loc[hourly["count"] == 60, "sum"]      # complete hours only
lam = hourly.mean()
print(f"Hours = {len(hourly)} |   = {lam:.3f} anomalies/hour")

# Exact & Cumulative probabilities
Y = stats.poisson(lam)
print(f"P(X = 5)  = {Y.pmf(5):.4f}")
print(f"P(X <= 3) = {Y.cdf(3):.4f}")
print(f"P(X >= 8) = {1 - Y.cdf(7):.4f}")

# Mean,Variance & Empirical check
print(f"Theory: mean = var =   = {lam:.3f}")
print(f"Sample: mean = {hourly.mean():.3f}, var = {hourly.var(ddof=0):.3f}")
```

```
Hours = 1666 |   = 5.350 anomalies/hour
P(X = 5)  = 0.1734
P(X <= 3) = 0.2193
P(X >= 8) = 0.1724
Theory: mean = var =   = 5.350
Sample: mean = 5.350, var = 4.941
```

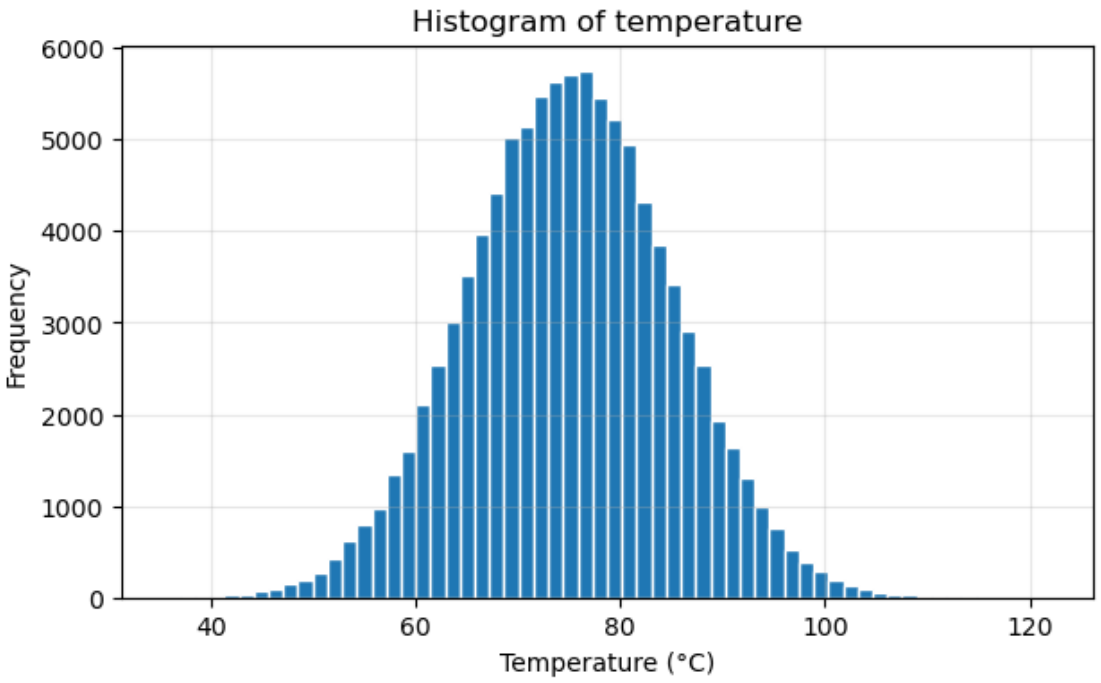
PART C:- Continuous Random Variables & Distributions

a) Selection & Summary of continuous variable

[14]:
```
t = df["temperature"]
print(f"Mean = {t.mean():.3f} | Variance = {t.var():.3f} | Std = {t.std():.3f}")

plt.hist(t, bins=60, edgecolor="white")
plt.xlabel("Temperature (°C)"); plt.ylabel("Frequency"); plt.title("Histogram of temperature")
plt.show()
```

```
Mean = 75.016 | Variance = 100.639 | Std = 10.032
```



b) Uniform Distribution

[16]:
```
h = df["humidity"]
a, b = h.min(), h.max()
U = stats.uniform(a, b - a)
print(f"a = {a}, b = {b} | Theory: mean = {(a+b)/2:.2f}, var = {(b-a)**2/12:.
↪2f} | Sample: mean = {h.mean():.2f}, var = {h.var():.2f}")

lo, hi = 40, 60
print(f"P({lo} <= H <= {hi}): theoretical = {U.cdf(hi) - U.cdf(lo):.4f} |␣
↪empirical = {h.between(lo, hi).mean():.4f}")
```

```
a = 30.0, b = 80.0 | Theory: mean = 55.00, var = 208.33 | Sample: mean = 55.00,
var = 208.45
P(40 <= H <= 60): theoretical = 0.4000 | empirical = 0.4012
```

c) Normal Distribution

[20]:
```
# Sustainability check
print(f"Skewness = {t.skew():.4f} (0 = symmetric) | Excess kurtosis = {t.kurt():
↪.4f} (0 = normal-like)")

# Parameter estimation and probabilities
mu, sigma = t.mean(), t.std()
Z = stats.norm(mu, sigma)
```

```
print(f"  = {mu:.2f} °C,   = {sigma:.2f} °C")
print(f"P(T < 60)      = {Z.cdf(60):.4f}  (empirical {np.mean(t < 60):.4f})")
print(f"P(T > 90)      = {1 - Z.cdf(90):.4f}  (empirical {np.mean(t > 90):.
 ↪4f})")
print(f"P(65 < T < 85) = {Z.cdf(85) - Z.cdf(65):.4f}  (empirical {np.mean((t >␣
 ↪65) & (t < 85)):.4f})")

# Z-scores
for label, idx in [("Hottest record", t.idxmax()), ("Coolest record", t.
 ↪idxmin()), ("Record #0", 0)]:
    v = t[idx]; z = (v - mu) / sigma
    print(f"{label:15s}: T = {v:7.2f} °C -> z = {z:+.2f} ({'unusual, |z|>3' if␣
 ↪abs(z) > 3 else 'typical'})")

# Empirical Rule (68 - 95- 99.7)
z = (t - mu) / sigma
```

```
Skewness = -0.0100 (0 = symmetric) | Excess kurtosis = 0.0138 (0 = normal-like)
  = 75.02 °C,   = 10.03 °C
P(T < 60)      = 0.0672  (empirical 0.0666)
P(T > 90)      = 0.0676  (empirical 0.0677)
P(65 < T < 85) = 0.6811  (empirical 0.6809)
Hottest record : T =  121.94 °C -> z = +4.68 (unusual, |z|>3)
Coolest record : T =   35.55 °C -> z = -3.93 (unusual, |z|>3)
Record #0      : T =   78.61 °C -> z = +0.36 (typical)
```

d) Exponential Distribution

[22]:
```
wait = df.loc[df.A, "timestamp"].diff().dropna().dt.total_seconds() / 60
lam_e = 1 / wait.mean()
E = stats.expon(scale=1 / lam_e)
print(f"Mean wait = {wait.mean():.2f} min |   = {lam_e:.4f} per min")

for tm in (5, 10, 30):
    print(f"t = {tm:2d} min: P(T<=t) = {E.cdf(tm):.4f} (emp {np.mean(wait <=␣
 ↪tm):.4f}) | "
          f"P(T>t) = {E.sf(tm):.4f} (emp {np.mean(wait > tm):.4f})")
```

```
Mean wait = 11.22 min |   = 0.0892 per min
t =  5 min: P(T<=t) = 0.3597 (emp 0.3735) | P(T>t) = 0.6403 (emp 0.6265)
t = 10 min: P(T<=t) = 0.5900 (emp 0.6109) | P(T>t) = 0.4100 (emp 0.3891)
t = 30 min: P(T<=t) = 0.9311 (emp 0.9417) | P(T>t) = 0.0689 (emp 0.0583)
```

PART D:- Distribution Analysis

a) Empirical CDF vs Normal CDF

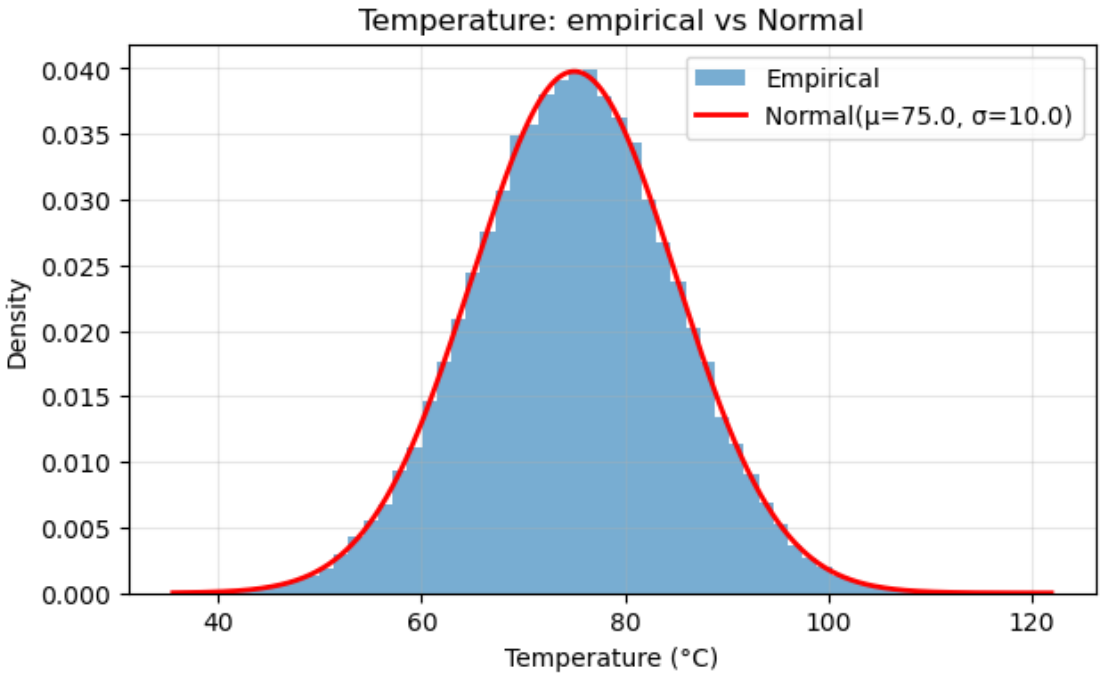
[31]:
```
pts = [55, 65, 75, 85, 95]
pd.DataFrame({"Empirical F(x)": [np.mean(t <= v) for v in pts],
              "Normal CDF": Z.cdf(pts)},
             index=[f"{v} °C" for v in pts]).round(4)
```

[31]:

	Empirical F(x)	Normal CDF
55 °C	0.0239	0.0230
65 °C	0.1592	0.1590
75 °C	0.4979	0.4994
85 °C	0.8404	0.8402
95 °C	0.9774	0.9768

PLot 1: Histogram vs Normal (PDF) -Temperature

[32]:
```
x = np.sort(t.values)
plt.hist(t, bins=60, density=True, alpha=0.6, label="Empirical")
plt.plot(x, Z.pdf(x), "r", lw=2, label=f"Normal( ={mu:.1f},  ={sigma:.1f})")
plt.xlabel("Temperature (°C)"); plt.ylabel("Density")
plt.title("Temperature: empirical vs Normal"); plt.legend()
plt.show()
```



Goodness of fit: summary

[34]:
```
gap = lambda a, b: np.max(np.abs(a - b))
pd.DataFrame({
```

```
    "Variable": ["Temperature", "Waiting time", "Waiting time"],
    "Model": ["Normal", "Exponential", "Geometric"],
    "Max CDF gap": [stats.kstest(t, Z.cdf).statistic, gap(emp, E.cdf(k)),␣
 ↪gap(emp, G.cdf(k))],
    "Model mean": [mu, E.mean(), G.mean()],
    "Data mean": [t.mean(), wait.mean(), wait.mean()],
}).round(4)
```

[34]:

	Variable	Model	Max CDF gap	Model mean	Data mean
0	Temperature	Normal	0.0023	75.0156	75.0156
1	Waiting time	Exponential	0.0213	11.2165	11.2165
2	Waiting time	Geometric	0.0048	11.2158	11.2165